



Transformers and Large Language Models(LLMs) Task

[Visit our website](#)

Introduction

For years, teaching computers to understand human language was one of the biggest challenges in computer science. Early systems relied on rigid grammatical rules or processed text one word at a time in a strict sequence. If a sentence was too long, the machine would literally "forget" how it started by the time it reached the end.

Everything changed with the invention of the transformer architecture. Instead of reading line by line, transformers can analyze an entire document all at once, capturing the deep contextual relationships between words regardless of how far apart they sit. This singular breakthrough paved the way for modern Large Language Models (LLMs) like GPT-4, Claude, and Llama.

As a developer, understanding the mechanics under the hood of a transformer will completely change how you build AI integrations. In this lesson, we will break down the architectural innovations behind language models and build a pure JavaScript simulation of the core algorithm that powers them: Self-Attention.

The Evolution of Natural Language Processing

To appreciate transformers, it helps to understand what came before them. Early neural networks processed language using sequential architectures. They read a sentence from left to right, updating an internal memory state word by word.

This created two massive engineering bottlenecks:

1. **The Vanishing Gradient / Memory Problem:** If a sentence was fifty words long, the influence of the first word would completely fade away by the time the network reached the final word. The system struggled with long-range dependencies, failing to connect a pronoun at the end of a paragraph to a proper noun at the very beginning.
2. **Lack of Parallelization:** Since the network had to process the first word before it could look at the second word, developers could not distribute the workload across thousands of modern graphics processing units (GPUs). Training large models was painfully slow.

The Transformer architecture solved both issues simultaneously by ditching sequential processing entirely and introducing parallel computation.

Core Architectural Innovations of the Transformer

The transformer relies on a series of distinct engineering systems working together to convert human language into mathematical relationships that a computer can process. These innovations allow modern Large Language Models (LLMs) to understand context, identify patterns, and generate coherent responses.

1. Tokenization and Embeddings

Computers cannot directly understand words, sentences, or meaning. When a prompt is submitted to an LLM, the text is first broken into smaller units called **tokens**. A token may represent an entire word, part of a word, a number, or even punctuation.

For example, the sentence:

"I love web development."

might be divided into tokens such as:

"I" | "love" | "web" | "development" | "."

After tokenization, each token is converted into an **embedding vector**, which is a large collection of numerical values. These vectors allow the model to represent semantic relationships mathematically. Words with similar meanings tend to have embeddings that are positioned closer together in the model's mathematical space.

For example, the words **developer**, **programmer**, and **engineer** may have embedding vectors that are relatively close to one another because they are frequently used in similar contexts. This process transforms human language into a format that can be processed by neural networks.

2. Positional Encoding

Unlike earlier language-processing systems that read text sequentially, transformers process all tokens simultaneously. While this greatly improves efficiency, it introduces a challenge: the model no longer inherently understands the order of words. Consider the following sentences:

"The dog bit the man."

and

"The man bit the dog."

Both sentences contain the same words, but the meaning is completely different because the word order has changed.

To solve this problem, transformers use **Positional Encoding**. Additional mathematical information is added to each token's embedding to indicate its position within the sequence. This allows the model to distinguish between words appearing at the beginning, middle, or end of a sentence.

As a result, the transformer can understand that changing word order can significantly alter meaning, even when the same vocabulary is used.

3. The Decoder and Next-Token Prediction

Many modern LLMs, including systems such as ChatGPT, are based on a **decoder-only architecture**. At their core, these models generate text by repeatedly predicting what token is most likely to appear next.

For example, given the prompt:

"The capital city of France is"

the model evaluates millions of learned language patterns and determines that the token *"Paris"* has a very high probability of being the next word.

Once that token is generated, it is added to the existing sequence:

"The capital city of France is Paris"

The model then predicts the next token again, such as a period or additional explanatory text. This process repeats continuously until a special stop token is produced or a predefined response length is reached.

This next-token prediction process is why LLMs are often compared to highly sophisticated autocomplete systems. However, because they have been trained on enormous amounts of text and can consider large amounts of context, they generate responses that appear intelligent, conversational, and highly relevant to the user's request.

Demystifying the Mathematics

What is a Vector?

If you do not have a background in advanced mathematics, seeing words described as lists of numbers can feel confusing. Let us break down exactly what a vector is in the context of web development and AI.

In JavaScript, a vector is simply an array of floating point numbers. When we say an embedding vector has four dimensions, it means the array contains exactly four numbers, like `[0.90, 0.10, 0.05, 0.00]`.

Each position, or index, in that array acts as a specific slider or score for a feature of meaning. For instance, imagine a model where:

- Index 0 tracks how related a word is to **Front-End Development**
- Index 1 tracks how related a word is to **Back-End Development**
- Index 2 tracks how related a word is to **Mobile Development**
- Index 3 tracks how related a word is to **Data Science**

A word like "React" might look like `[0.90, 0.10, 0.05, 0.00]`. This tells the computer that the word is heavily associated with front-end code, slightly related to back-end tools, and completely unrelated to data science. Representing language as rows of numbers allows us to use simple code loops to calculate how concepts relate to each other.

What is a Dot Product?

The most important mathematical operation used inside a language model is the **Dot Product**. While it sounds intimidating, the actual code is straightforward arithmetic that you likely use every day.

The dot product is a way to combine two arrays of numbers of equal length to get a single total score. To calculate a dot product, you multiply the numbers that sit in matching positions across both arrays, and then add all those individual multiplication results together.

Let us trace a manual example using two mock three-dimensional arrays:

- Array A = `[2, 4, 3]`
- Array B = `[3, 1, 5]`

To find their dot product:

1. Multiply the numbers at index 0: $2 \times 3 = 6$
2. Multiply the numbers at index 1: $4 \times 1 = 4$
3. Multiply the numbers at index 2: $3 \times 5 = 15$
4. Add all the results together: $6 + 4 + 15 = 25$

The dot product of these two arrays is 25. In machine learning, we use this exact operation as a similarity calculator. If two vectors point in a similar conceptual direction, their matching indices will both contain high numbers, resulting in a large dot product score. If the vectors are completely unrelated, their matching positions will contain numbers close to zero, resulting in a tiny dot product score.

The Superpower of Transformers: Self-Attention

The absolute crown jewel of the transformer is an algorithm called Self-Attention. It allows the model to look at a specific word in a sentence and mathematically calculate how much attention it should pay to every other word in that same sentence. Consider this classic linguistic puzzle:

"The animal did not cross the street because it was too tired."

As human readers, we instantly know that the word "it" refers to the animal.

Now change just one word:

"The animal did not cross the street because it was too wide."

Suddenly, the word "it" refers to the street.

Sequential systems completely stumbled on this subtle context mapping. Self-attention solves it by scoring the mathematical relationship between tokens. When processing "it", the algorithm calculates similarity scores against all other words. In the first sentence, the mathematical calculation yields a high attention weight for "animal". In the second sentence, the attention weight shifts dynamically to "street".

The Self-Attention Mechanics

How does this work mathematically in code? The self-attention mechanism converts every single token vector into three distinct vectors using matrix multiplication:

1. **Query (Q):** What am I looking for? (Think of this as a search query term).
2. **Key (K):** What relevance tags do I offer? (Think of this as a video title or tag in a database index).

3. **Value (V):** What actual content do I hold if I am relevant? (Think of this as the actual video content).

To compute the attention score, the system takes the Query vector of the current word and calculates the Dot Product against the Key vectors of every word in the sentence. The dot product is a straightforward mathematical operation where you multiply matching indices of two arrays and sum up the results.

A higher dot product means the two words are highly relevant to each other. The resulting raw numbers are converted into percentages that sum up to 1, and these percentages are multiplied by the Value vectors to construct a highly context-aware output vector.

Simulating Self-Attention in JavaScript

Here is a clean, raw JavaScript script simulating how a model calculates attention weights for a single token against a small sentence structure. All logic lives sequentially within a single execution file to ensure it remains clear and readable.

```
// A tokenized sentence array: "The", "AI", "Assistant"
// Each token has a mock 3-dimensional Query, Key, and Value vector
const tokens = [
  { text: "The",      query: [0.1, 0.2, 0.0], key: [0.1, 0.1, 0.3], value:
[10, 0, 0] },
  { text: "AI",      query: [0.8, 0.9, 0.1], key: [0.7, 0.8, 0.2], value:
[0, 20, 0] },
  { text: "Assistant", query: [0.7, 0.8, 0.2], key: [0.6, 0.9, 0.1], value:
[0, 0, 30] }
];
// Let's compute what the token "AI" (index 1) should pay attention to
const targetToken = tokens[1];
console.log(`Calculating self-attention mapping for token:
"${targetToken.text}"\n`);
// Function to compute the dot product of two vector arrays
function calculateDotProduct(vectorA, vectorB) {
  let product = 0;
  for (let i = 0; i < vectorA.length; i++) {
    product += vectorA[i] * vectorB[i];
  }
  return product;
}
// Step 1: Calculate raw attention scores by matching AI's Query against all
Keys
const rawScores = tokens.map(token => {
  const score = calculateDotProduct(targetToken.query, token.key);
```

```
    return { text: token.text, rawScore: score };
  });
  console.log("Raw Dot Product Attention Scores:");
  console.dir(rawScores);
```

While the above code looks like standard JavaScript logic, the script performs the exact same foundational steps that a Large Language Model uses to process human text under the hood.

1. Defining the Language Tokens

The script begins with an array of objects called `tokens`. This acts as a miniature three-word sentence: "The", "AI", and "Assistant". To match the structural design of a real transformer model, each word object holds three distinct arrays of numbers representing its **Query (Q)**, **Key (K)**, and **Value (V)** vectors.

- **The Query (Q):** Represents what a specific word is actively searching for in the sentence to understand its own context.
- **The Key (K):** Represents the relevance tags or characteristics that a word offers to other words looking at it.
- **The Value (V):** Holds the actual informational content of the word, which gets passed along to the final output if the word turns out to be relevant.

2. Isolating the Target Word

Next, the code isolates a single focal point for the calculation. By selecting `tokens[1]`, the script assigns the word "AI" as the `targetToken`. The goal of the rest of the script is to answer a singular question: *Based on what the word "AI" is looking for (its Query vector), how much attention should it pay to every word in this sentence (including itself)?*

3. The Relevance Calculator (The Dot Product)

To determine how well the words match up, the code defines the `calculateDotProduct` function. This function acts as a mathematical similarity engine.

It loops through two arrays at the exact same time, multiplies the numbers that sit in matching index positions (index 0 times index 0, index 1 times index 1, and so on), and keeps adding those multiplication results to a running total. In machine learning, if two lists of numbers have high values in the exact same positions, their dot product total climbs significantly. If they do not match up well, the total stays close to zero.

4. Generating the Attention Report

Finally, the script uses the built-in JavaScript `.map()` method to loop over every word in the sentence. For each word, it passes the **Query vector of "AI"** and the **Key vector of the current word** into the dot product calculator.

When you run this code in Node, the terminal will print out three raw attention scores:

- **"AI" Query** matched against **"The" Key**:
 - Yields a low score (**0.20**) because a background article like "The" carries very little semantic relationship to the technical term "AI".
- **"AI" Query** matched against **"AI" Key**:
 - Yields a high score (**1.30**) because a word is naturally highly relevant to itself.
- **"AI" Query** matched against **"Assistant" Key**:
 - Yields an equally high score (**1.30**) because the context lines of "AI" and "Assistant" are mathematically designed to lock together.

By looking at these final numbers, you can see the exact mathematical framework a transformer uses to dynamically calculate language weights and prioritize context. When you execute the above in Node, you will see that the token "AI" yields a significantly higher dot product score when paired with the key vectors for "AI" and "Assistant" than it does for the baseline article "The". This proves how transformer mechanics dynamically isolate contextual relationships.

Bringing It All Together: From Prompt to Prediction

At first glance, transformers and Large Language Models can seem incredibly complex. However, when we break the architecture into individual components, we can see that each part solves a specific problem in language processing. When a user enters a prompt into an LLM, the process begins with **tokenization**, where the text is split into smaller pieces called tokens. These tokens are then converted into **embedding vectors**, allowing the model to represent words and concepts as mathematical values rather than plain text.

Because transformers process all tokens simultaneously, the model must also understand where each token appears within the sequence. This is achieved through **positional encoding**, which adds information about word order while preserving semantic meaning.

Once the tokens and positions have been prepared, the transformer applies its most important innovation: **self-attention**. Each token compares itself to every other token in the sequence using Query, Key, and Value vectors. Through a series of dot product calculations, the model determines which words are most relevant to understanding the current context.

For example, when processing the sentence:

"The AI assistant generated code because it was asked to."

the model can use attention mechanisms to determine that the word "*it*" most likely refers to the *AI assistant* rather than the *code*. This ability to dynamically connect related concepts across a sentence is one of the key reasons transformers outperform earlier language-processing architectures. After context has been established through multiple attention layers, the model enters the **next-token prediction** stage. Based on everything it has learned from the prompt and its training data, it predicts the most probable next token. That token is added to the sequence, and the process repeats until a complete response has been generated.

As web developers, it is important to understand that modern AI systems are not retrieving prewritten answers from a database. Instead, they are continuously performing mathematical calculations over vectors, attention scores, and probabilities to generate responses one token at a time. Understanding these foundational concepts provides the knowledge needed to work effectively with AI-powered applications. Whether you are integrating an LLM into a web application, building AI-assisted user experiences, or consuming AI APIs, every interaction ultimately relies on the transformer mechanisms explored in this lesson.



Spot check 1

Let's see what you can remember from this section.

1. Why did older sequential language models struggle to process long documents or paragraphs?
2. What is the fundamental purpose of Positional Encoding within the transformer execution loop?
3. In the self-attention mechanism, which two vectors are multiplied together via a dot product to calculate a word's raw attention weight score?



Take note

The task below is **auto-graded**. An auto-graded task still counts towards your progression and graduation. Give it your best attempt and submit it when you are ready.

When you select “Request Review”, the task is automatically complete, you do not need to wait for it to be reviewed by a mentor.

You will then receive an email with a link to a model answer, as well as an overview of the approach taken to reach this answer.

Take some time to review and compare your work against the model answer. This exercise will help solidify your understanding and provide an opportunity for reflection on how to apply these concepts in future projects.

Once you've done that, feel free to progress to the next task.



Auto-graded task

In this practical activity, you will move beyond a single target calculation and build a complete context processor engine using Node.js. Your program will parse an expanded dictionary array of token blocks, generate dynamic dot product attention scores across all items, implement a normalization scaling utility, and build an integrated context vector output summary.

1. Initialize Your Workspace File
 - Open the JavaScript file inside your assignment submission folder named **attention_processor.js** where you will find the Token Vector Catalog.
2. Implement the Attention Calculations and Value Weighting. Your script must implement the following algorithmic operations below the variables:
 - Loop over the entire text sequence catalog array and compute the raw attention score by calculating the dot product of the Query vector of the word "State" against the Key vector of each individual token in the sequence. Save these scores as an array of objects.

- To mimic how real transformers prevent raw scores from scaling out of proportion, implement a soft normalization scaling wrapper directly on your calculated output loop.
 - Add a new property named `scaledScore` to your tracking objects by dividing each raw dot product score by 2 (which acts as the square root scaling factor for our 4-dimensional vector space).
- Print your raw and scaled scoring calculations directly to the terminal view using `console.log()` to verify accuracy.



Code hack

To create a clean, non-destructive data collection mapping, use JavaScript's built-in `.map()` method to loop over the `textSequence`. Your tracking objects should return the word name along with your calculated `rawScore` and `scaledScore` keys so the automated grading compiler can locate your results effortlessly.

Once your processing code loops are fully coded, run your file through your local console pane using **Node.js**:

```
node attention_processor.js
```

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.



Spot check 1 answers

1. Older sequential networks processed text word by word, which caused long-range dependencies and contextual meanings to fade away over long paragraph intervals.
 2. Positional Encoding applies a mathematical wave profile to token embeddings to reintroduce word sequence information, since transformers analyze all terms simultaneously.
 3. The Query vector of the focus token is multiplied by the Key vector of every token in the sequence.
-



Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.
